

JavaScript 함수 기초

학습 체크리스트 (1/2)

☐ 함수 선언문(`function`)과 함수 표현식(`const fn = function`) — 문법 차이 및 호이스팅 유무

☐ 화살표 함수(`=>`) — 중괄호·return·소괄호 생략 조건

☐ 매개변수(Parameter)와 인자(Argument) — 역할 구분 및 데이터 전달 흐름

학습 체크리스트 (2/2)

☐ 기본값 매개변수(`param = 기본값`) — 누락 인자 방어 패턴

☐ 나머지 매개변수(`...rest`) — 가변 인자 배열 수집

☐ 구조 분해 할당 — 객체 `{ key }` / 배열 `[a, b]` 적용

함수란?

함수 (Function)

여러 명령을 하나로 묶어 이름을 부여하고, 필요할 때 언제든지 호출해 재사용할 수 있는 코드 블록

- **묶기:** 반복되는 코드를 하나의 이름 아래로 정리
- **재사용:** 같은 로직을 여러 곳에서 이름만 불러 실행
- **유지보수:** 변경 사항을 한 곳에서만 수정하면 전체 반영

함수 선언문 — 개념

함수 선언문 (Function Declaration)

`function` 키워드로 시작해 이름을 부여하는 전통적인 방식

- **호이스팅 적용:** 코드 실행 전 함수 전체가 스코프 최상단으로 끌어올려짐
- **선언 이전 호출 가능:** 함수 정의보다 먼저 호출해도 정상 동작
- **대표 사용처:** 독립적으로 재사용되는 일반 함수 정의

함수 선언문 — 코드 예시

```
1 // 함수 정의 이전에 호출해도 호이스팅 덕분에 정상 동작
2 console.log(greet("지민")); // "안녕하세요, 지민님!"
3 console.log(greet("유나")); // "안녕하세요, 유나님!"
4 console.log(greet("민준")); // "안녕하세요, 민준님!"
5
6 // function 키워드 + 이름 + 매개변수 + 본문
7 function greet(name) {
8     return `안녕하세요, ${name}님!`;
9 }
```

함수 표현식 — 개념

함수 표현식 (Function Expression)

이름 없는 함수를 변수에 값처럼 담아 선언하는 방식

- **호이스팅 미적용:** 함수 본체는 런타임 시점에 할당됨
- **선언 이전 호출 불가:** 호출 시 `TypeError` 발생
- **대표 사용처:** 콜백 함수, 조건부 할당, 스코프 제한

함수 표현식 — 코드 예시

```
1  // const 변수에 익명 함수를 값으로 할당
2  const square = function (num) {
3      return num * num;
4  };
5
6  console.log(square(4)); // 16
7  console.log(square(7)); // 49
8  console.log(square(9)); // 81
9
10 // 선언 이전에 호출하면 TypeError 발생!
11 // console.log(square(5)); // ← 위로 올리면 오류
```


선언문 vs 표현식 — 비교

구분	함수 선언문	함수 표현식
선언 이전 호출	✅ 가능	❌ TypeError
호이스팅 범위	함수 전체	변수 선언만
이름 부여	필수	선택 (익명 가능)
주 사용처	독립 함수	콜백, 스코프 제한

화살표 함수 — 개념

화살표 함수 (Arrow Function)

ES6에서 도입된 `=>` 기호를 사용하는 함수 표현식의 간결 단축 문법

- 기본 형태: `const fn = (매개변수) => { return 값; }`
- 종괄호·return 생략: 본문이 반환식 하나뿐이면 생략 가능
- 소괄호 생략: 매개변수가 단 하나이면 소괄호도 생략 가능

화살표 함수 — 축약 예시

```
1  // 일반 함수 표현식
2  const double = function (n) { return n * 2; };
3
4  // 중괄호 + return 생략
5  const double2 = (n) => n * 2;
6
7  // 소괄호까지 생략 (완전 축약)
8  const double3 = n => n * 2;
9
10 console.log(double(5)); // 10
11 console.log(double3(5)); // 10
```

매개변수와 인자 — 개념

매개변수 (Parameter)

함수를 정의할 때 소괄호 안에 선언하는 변수로, 외부에서 전달될 데이터를 안으로 수집하는 매개체

인자 (Argument)

함수를 호출할 때 괄호 안에 실제로 넣어 전달하는 구체적인 값 또는 데이터 자체

- **수집과 전달:** 매개변수는 데이터를 받는 상자, 인자는 넣는 물건
- **이름과 값:** 매개변수는 변수의 이름이며, 인자는 실재하는 값

매개변수와 인자 — 코드 예시

```
1  // num은 전달되는 값을 수신하기 위해 정의된 '매개변수 (Parameter)'  
2  function triple(num) {  
3      return num * 3;  
4  }  
5  
6  // 9는 함수 실행 시 매개변수에 주입하는 '인자 (Argument)'  
7  const result = triple(9);  
8  
9  console.log(result); // 27
```

기본값 매개변수 — 개념

기본값 매개변수 (Default Parameter)

인자가 전달되지 않아 `undefined` 가 될 경우를 대비해, 매개변수 선언 시 미리 기본값을 지정하는 방어 패턴

- 문법: `function fn(param = 기본값) {}`
- 동작: 인자가 없으면 기본값이, 있으면 전달된 값이 사용됨

기본값 매개변수 — 예시

```
1 function introduce(name = "익명", role = "방문객") {  
2     return `이름: ${name}, 역할: ${role}`;  
3 }  
4  
5 console.log(introduce("철수", "관리자"));  
6 // 이름: 철수, 역할: 관리자  
7  
8 console.log(introduce());  
9 // 이름: 익명, 역할: 방문객
```

나머지 매개변수 — 개념

나머지 매개변수 (Rest Parameter)

`...` 점 세 개를 매개변수 앞에 붙여, 나머지 인자들을 하나의 배열로 수집하는 기능

- **위치 제약:** 매개변수 목록의 **마지막** 위치에만 사용 가능
- **`arguments`와의 차이:** `arguments` 는 유사 배열, `rest` 는 진짜 배열

나머지 매개변수 — 예시

```
1 function sumAll(first, ...rest) {  
2   console.log("첫 번째 값:", first); // 10  
3   console.log("나머지 묶음:", rest); // [20, 30, 40]  
4   return rest.reduce((acc, val) => acc + val, first);  
5 }  
6  
7 console.log(sumAll(10, 20, 30, 40)); // 100
```

구조 분해 할당이란?

구조 분해 할당 (Destructuring Assignment)

배열이나 객체의 데이터를 개별 변수에 꺼내어 바인딩하는 ES6 단축 문법

- 목적: 중복 접근 코드 제거, 가독성 향상
- 객체 구조 분해: `const { key } = obj`
- 배열 구조 분해: `const [a, b] = arr`

객체 구조 분해 — 개념

- 문법: `const { 속성명 } = 객체` — 키 이름과 동일한 변수명으로 추출
- 별칭 지정: `{ key: 새이름 }` 형태로 다른 변수명 바인딩 가능
- 기본값 결합: `{ key = 기본값 }` 형태로 없을 때 대비책 지정 가능

객체 구조 분해 — 예시

```
1  const product = { name: "노트북", price: 1500000 };
2
3  const { name, price } = product;
4  console.log(name); // "노트북"
5  console.log(price); // 1500000
6
7  // 별칭 지정: name → productName으로 바인딩
8  const { name: productName } = product;
9  console.log(productName); // "노트북"
```

배열 구조 분해 — 개념

- 문법: `const [변수1, 변수2] = 배열` — 인덱스 순서대로 바인딩
- 건너뛰기: 쉼표(,)만으로 불필요한 자리를 비워 건너뛸 수 있음
- 스왑 패턴: 임시 변수 없이 두 변수 값을 교환 가능

배열 구조 분해 — 예시

```
1  const scores = [95, 82, 74, 60];
2
3  // 세 번째 자리(74) 건너뛰기
4  const [first, second, , fourth] = scores;
5  console.log(first); // 95
6  console.log(fourth); // 60
7
8  // 스왑 패턴 (임시 변수 불필요)
9  let a = 1, b = 2;
10 [a, b] = [b, a];
11 console.log(a, b); // 2 1
```